

T4: Second Iteration

 Published

 Assign To

 Edit



Team Assignment 4: Second Iteration

This is a very dense assignment, you are responsible for reading this specification in its entirety.

Overview

Implement the final version of your service. If it does not include (at least) the full API as described in your proposal, your README should explain what is different and why you decided to change it. If you added anything significant to the API that was not described in your revised proposal, explain what and why.

Develop a **demoable** client that uses your service API and targets one or more of the designated target audiences. For your reminder here are the acceptable target audiences:

- Elderly (defined for this purpose as age 75 or over)
- Disabled or impaired (e.g., vision, hearing, speech, motor, cognitive)
- Healthcare workers (doctors, medical technicians, nurses, aides, etc.)
- First responders (police, fire, EMT)
- Indigenous peoples ("distinct social and cultural groups that share collective ancestral ties to the lands and natural resources where they live, occupy or from which they have been displaced")
- Homeless people ("lacks a fixed, regular, and adequate nighttime residence")
- Potentially others, needed to be approved in advance! (please link/paste approval in README)

Make sure your service can interface to *multiple instances* of the client running concurrently and tell them apart. We would prefer if your client app lived in a separate repository to your service but it is okay if it is in the same repository ([Google stores billions of lines of code in a single repository](https://cacm.acm.org/magazines/2016/7/204032-why-google-stores-billions-of-lines-of-code-in-a-single-repository/fulltext)  (<https://cacm.acm.org/magazines/2016/7/204032-why-google-stores-billions-of-lines-of-code-in-a-single-repository/fulltext>)). If it is in different repositories make sure the repositories link to one another in the README.

Testing

Update your unit test suite to support all the major "units" (methods, functions, procedures, subroutines, etc.) in your code. You should use a mocking framework if applicable. Unit tests should be grouped so related code (e.g., classes, modules, files, etc.) share setup, teardown and mocks when appropriate. Your entire collection of unit tests should run automatically during CI.

Update your API (system) test suite to exercise your service's complete API, e.g., using some API testing tool. Your tests should exercise the multiple clients and persistent data aspects of your service. Your API tests should also run automatically during CI.

Implement internal integration tests between all assemblies of multiple units within your service that work together in some way, either by invoking each other or by sharing data. Add external integration tests checking how your code uses any external resources like files and databases and any third-party code like libraries and services. Document each test to explain what it is integrating with. All your integration tests should run automatically during CI.

Implement end-to-end tests where your client exercises as much functionality of your service as possible. Although ideally automated, it's ok to run end-to-end tests manually. Document any manual tests with a checklist or some other mechanism to make sure you can re-run the exact same set of tests as needed, and include these instructions in your repo.

It is important to make sure that you test using a large array of different kinds of values both valid and invalid to ensure proper functionality. Do not just test the happy path and call it a day.

Static Analysis + Branch Coverage

At this point you are responsible for performing Static Analysis on your codebase(s) as well as branch coverage. You need to try to fix all bugs found by the static analysis tool of your choice, you can ignore the lowest level of static bugs (as described in the documentation for your tool) If static bugs remain at the time of submission you must document these and explain why they remain. You will also need to use a branch coverage tool to ensure that you have 85% coverage along your codebase'(s) branches. Both of these are mandatory for the service and optional for the client.

Continuous Integration

Just like you did during the mini project at the beginning of the semester, you will need to implement a continuous integration loop on your codebase(s). In this loop you will need to do the following:

1. Run all tests (Unit tests, Integration tests, API tests etc)
2. Run checkstyle, generate a report and place it in your repository
3. Run static analysis, generate a report and place it in your repository
4. Run branch coverage, generate a report and place it in your repository

You will need to include some CI reports in your codebase but you can place/link to these manually if you want. This is mandatory for the service, optional for the client (but recommended).

Documentation

You will need to maintain and update several pieces of information that contain documentation.

Update your README to describe what your client app does, how it uses your service, and how it helps its target audience. Explain what its users can do now with your app that they could not do before or better/faster/cheaper than they could before.

Also add to your README.md instructions explaining how to build, run and test your sample client with your service. All configuration files needed for build, run and test must be included in your repo. Describe what a third-party would need to do to develop their own client that uses your service.

Your repository's README.md should document all the operational entry points to your service and/or link to some external API documentation you wrote. Cover all the inputs and outputs, including status codes. If certain entry points must be called in certain orders or never called in certain orders, make sure to say so. Your README should also include instructions explaining how to build, run and test your service. All configuration files needed for build, run and test must be included in your repo.

If any third-party code is included in your codebase, your README should also document exactly which code this is, where it resides in your repository, and where you got it from (e.g., download url). However, in most cases you should use a package manager or build system to fetch third-party libraries as needed rather than bloating your codebase.

Continue using your project management tool to keep track of tasks completed by different members of your team. Also continue to document the non-trivial code you write using javadoc (or equivalent) You should also include a link to your project management tool's board in your README as well!

You will need to provide information regarding the cloud deployment of your service + client app in your README.md as well.

Submission

Make sure all your test code and scripts have been committed to the main branch of your repository (its ok to have some other test code in branches but it will not be considered part of your submission). All main branch commit messages, except for the initial repo-setup commit(s), should say something meaningful about what was added/removed/changed. Your entire codebase, including test cases, should be compliant with a style checker appropriate for your language/platform. Include some recent reports from your style checker in your repository. Make sure to [tag](https://stackoverflow.com/questions/18216991/create-a-tag-in-a-github-repository)  [the set of file revisions](https://stackoverflow.com/questions/18216991/create-a-tag-in-a-github-repository) considered part of the second iteration.

One member of your team should submit a url pointing to your codebase repository. We encourage you to make your repository public, but if you prefer to keep it private then make sure the instructor and at

least your project mentor have access! Below is a checklist to ensure that you have everything you need for this assignment, the checklist is at a high level, see appropriate sections for details!

Checklist

- Finalized Service (feature wise)
- Implemented demoable client app for a target group
- Implemented testing: integration, system, end-to-end etc
- Achieved 85% branch coverage
- Fixed bugs from Static Analysis
- Implemented CI Loop
- Updated Documentation

If you prefer to have a copy of this document, you may access a PDF version in the Files > Team Assignments > Specs > T4.pdf

Points 75

Submitting a website url

Due	For	Available from	Until
Nov 27, 2024	Everyone	Oct 11, 2024 at 12am	Dec 1, 2024 at 11:59pm

+ Create Rubric

🔍 Find Rubric